Brute-Force Login Attempt Detector

Project Report

Domain: Cybersecurity

Name	ADWAID K.P
Programming Language	Python 3
Date	23 October 2026

1. Introduction

Passwords are the most common way to protect online accounts, and attackers often try to break them by guessing. A brute-force attack is one where an attacker repeatedly tries different passwords on an account until the correct one is found. Such attacks leave a clear trace in login logs: many failed attempts from the same source in a very short time.

This project builds a simple detector in Python that reads login records, finds this pattern, and raises an alert for the security team.

2. Objective

• To understand how brute-force login attacks appear in log data.

• To build a tool that automatically detects suspicious IP addresses.

• To warn when an attack may have succeeded (a successful login after many failures).

• To present results as a clear, readable report.

3. Problem Statement

Servers record thousands of login attempts every day. Reading these logs by hand to spot an attack is slow and error-prone. A security analyst needs an automatic way to flag IP addresses that make too many failed logins within a short period, and to know whether any of them eventually got in.

4. Tools and Technologies

Tool	Purpose
Python 3	Main programming language
csv module	Reading and writing the log file
datetime module	Handling timestamps and time windows
collections (defaultdict)	Grouping login attempts by IP address
random module	Generating sample log data

No machine learning and no external libraries are used. The project runs on the Python standard library only.

5. Data Source

The project generates its own sample log file, **login_logs.csv**, so no real user data is needed. Each record has four fields: **timestamp**, **ip**, **username** and **status** (SUCCESS or FAILED). The sample contains 43 entries: 30 normal login events from three internal IP addresses, and 13 events from one attacker IP (203.0.113.45): 12 failed logins on the admin account, followed by one successful login.

6. Methodology

The detection uses a simple rule-based, sliding-window approach:

• **Load logs:** Read every record from the CSV file.

• **Group by IP:** Store the times of failed logins and successful logins separately for each IP address.

• **Sliding window:** For each IP, start at each failed login and count how many failures occur within the next 60 seconds.

• **Threshold check:** If the count reaches 5 or more, the IP is flagged as a brute-force source.

• **Compromise check:** If the same IP has a successful login after the failed attempts, the account is marked as possibly compromised.

• **Report:** Print the total entries analysed, the rule used, and details of each alert.

The threshold (**MAX_FAILS = 5**) and window (**WINDOW_SECONDS = 60**) are configurable at the top of the program.

7. System Design

```
Login Logs (CSV)
      |
      v
 Read and parse records
      |
      v
 Group by IP address
      |
      v
 Sliding 60-second window
      |
      v
 5+ failures? --No--> Normal
      | Yes
      v
 Success after failures?
      |
      v
 Alert and summary report
```

8. Implementation

The program is in a single file, **app.py**, with four functions:

Function	Description
generate_sample_logs()	Creates login_logs.csv with normal users and one attacker
load_logs()	Reads the CSV into a list of records
detect_bruteforce()	Applies the sliding-window rule and returns alerts
main()	Runs the steps and prints the final report

Core detection logic:

```
for ip, times in fails.items():
    times.sort()
    for i in range(len(times)):
        window = [t for t in times[i:]
                  if (t - times[i]).total_seconds() <= WINDOW_SECONDS]
        if len(window) >= MAX_FAILS:
            compromised = any(s > window[-1]
                              for s in successes.get(ip, []))
            alerts.append({...})
            break
```

9. Results

The program was run on the sample log. Output:

```
==================================================
 BRUTE-FORCE LOGIN ATTEMPT DETECTOR
==================================================
Total log entries analysed: 43
Rule: 5+ failed logins within 60 seconds

[ALERT] IP 203.0.113.45
   Failed attempts : 12
   First failure   : 2026-10-23 10:02:00
   WARNING: successful login after failures. Account may be compromised!
```

• The attacker IP 203.0.113.45 was correctly detected (12 failed logins, 3 seconds apart).

• The normal internal IPs (192.168.1.10, 192.168.1.11, 10.0.0.5) were **not** flagged, so there were no false alarms on normal activity.

• The tool also detected the successful login after the failures, which is the most dangerous situation.

10. Limitations

• It uses sample data, not live server logs.

• A slow attack (a few attempts spread over hours) will not be caught by a 60-second window.

• Attackers using many different IP addresses (distributed attacks) are not linked together.

• The tool only raises alerts; it does not block the attacker automatically.

11. Future Scope

• Read real server logs such as SSH or web server logs.

• Automatically block flagged IPs using a firewall rule.

• Send email or SMS alerts to administrators.

• Add a dashboard with charts of attack activity.

12. Conclusion

This project shows that a brute-force login attack can be detected with a simple rule: too many failed logins from one IP within a short time. The detector successfully identified the attacker in the sample data, ignored normal users, and warned about a possible account compromise. Although basic, the same idea is used in real tools such as intrusion detection systems and login protection services, and it can be extended to work with real logs and automatic responses.

13. References

• Python documentation: https://docs.python.org/3/

• OWASP: Blocking Brute Force Attacks, https://owasp.org/www-community/controls/Blocking_Brute_Force_Attacks